# Cubic Regularization Methods: From Classical Theory to Modern Large-Scale Applications

## A Survey and Experimental Analysis

Your Name

Supervised by: [Professor Name]

Department of Mathematics
University Name

April 2026

# Outline


1. Introduction
2. Related Work
3. Methodology and Preliminary Results
4. Conclusion and Future Work

# The Problem: Limitations of Classical Methods

## Why Cubic Regularization?

**Newton's Method Issues:**

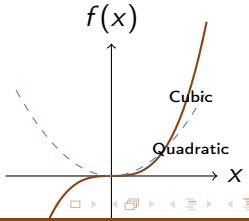
- No global convergence guarantees
- Diverges if Hessian is singular/indefinite
- Attracted to saddle points in non-convex problems
- Requires line search or trust-region modifications

**First-Order Methods:**

- Slow convergence on ill-conditioned problems
- Get stuck at saddle points (problematic for ML)
- $\mathcal{O}(\epsilon^{-2})$ complexity for non-convex

**The Cubic Model**

$$m_k(s) = f(x_k) + \nabla f(x_k)^\top s + \frac{1}{2} s^\top \nabla^2 f(x_k) s + \frac{M}{6} \|s\|^3$$



# Historical Context and Recent Advances

## Why Now?

**Classical Era (2006-2011):**

- Nesterov & Polyak (2006): Cubic Regularization
- Cartis, Gould, Toint (2011): Adaptive framework (ARC)
- Theory: $\mathcal{O}(\epsilon^{-3/2})$ complexity for non-convex

**Modern Era (2020-2024):**

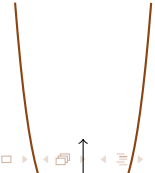
- **Scalability**: Subspace methods, lazy Hessians
- **Stochastic**: Sub-sampling for finite-sum problems
- **Deep Learning**: AdaCubic (2024) - first DL applications

**Key Insight**

Cubic regularization provides **second-order stationarity**:

$$\nabla f(x) = 0, \quad \nabla^2 f(x) \succeq 0$$

Escapes strict saddle points automatically!



# Objectives of This Survey

## Scope and Contributions

1. **Theoretical Analysis**
   - Complexity bounds and convergence properties
   - Comparison: CR vs. Gradient Descent vs. Quasi-Newton
2. **Algorithmic Variants**
   - Adaptive CR (ARC) with dynamic parameter selection
   - Stochastic and subspace methods for large-scale problems
3. **Experimental Evaluation**
   - Implementation on LIBSVM datasets (a9a, covtype)
   - Logistic regression and non-convex regularization
   - Convergence analysis and computational efficiency

**Research Question**

Can modern cubic regularization variants achieve practical scalability while maintaining theoretical guarantees for second-order stationarity?

# Theoretical Foundations

## From Nesterov-Polyak to Adaptive Frameworks

### The Cubic Regularization Subproblem

At iteration $k$, solve:

$$s_k \in \arg\min_{s \in \mathbb{R}^d} \left\{ \langle \nabla f(x_k), s \rangle + \frac{1}{2} s^\top \nabla^2 f(x_k) s + \frac{M}{6} \|s\|^3 \right\}$$

**Key Theoretical Results:**

| Method | Complexity | Hessian | Reference |
|---|---|---|---|
| Classical CR | $\mathcal{O}(\epsilon^{-3/2})$ | Exact | Nesterov & Polyak (2006) |
| ARC | $\mathcal{O}(\epsilon^{-3/2})$ | Exact/Inexact | Cartis et al. (2011) |
| Sub-sampled CR | $\mathcal{O}(\epsilon^{-3/2})$ w.h.p. | Sampled | Kohler & Lucchi (2017) |
| SSCN | $\mathcal{O}(\epsilon^{-3/2})$ | Subspace | Zhao et al. (2024) |

*Note: All achieve optimal second-order oracle complexity for non-convex Lipschitz-Hessian*

# The Adaptive Cubic Regularization (ARC) Framework

Cartis, Gould, and Toint (2011)

**Adaptive Mechanism:**

- Replace fixed $M$ with adaptive $\sigma_k$
- Compute ratio: $\rho_k = \frac{\text{actual reduction}}{\text{predicted reduction}}$
- Update $\sigma_k$ based on $\rho_k$:

$$\sigma_{k+1} = \begin{cases} \gamma_0 \sigma_k & \text{if } \rho_k < \eta_0 \text{ (unsuccessful)} \\ \sigma_k & \text{if } \eta_0 \leq \rho_k < \eta_1 \\ \gamma_1 \sigma_k & \text{if } \rho_k \geq \eta_1 \text{ (very successful)} \end{cases}$$

**Algorithm 1** ARC Framework

```
Initialize x_0, σ_0 > 0
for k = 0, 1, 2, ... do
    Compute s_k (approx.) solving:
    min_s m_k(s)
    Compute ρ_k
    if ρ_k ≥ η_0 then
        x_{k+1} = x_k + s_k
    else
        x_{k+1} = x_k
    end if
    Update σ_{k+1}
end for
```

**Advantages:** Automatic adjustment to local problem geometry; no manual tuning of $M$

# Modern Variants for Large-Scale ML

## Recent Advances (2020-2024)

**Stochastic Subspace Cubic Newton (SSCN):**

- Random coordinate subspace $S \in \mathbb{R}^{d \times \tau}$
- Reduced subproblem: dimension $\tau \ll d$
- Complexity: $\mathcal{O}(\epsilon^{-3/2})$ independent of $d$
- Per-iteration: $\mathcal{O}(d\tau^2)$ vs. $\mathcal{O}(d^3)$

**Lazy Hessian Methods:**

- Reuse Hessian across iterations
- Update only when necessary
- Significant computational savings

**AdaCubic (2024):**

- First scalable CR for deep learning
- Hutchinson's method for Hessian approximation
- Dynamic cubic weight adjustment
- Tested on CV, NLP tasks

**Sub-sampled CR:**

- Sample gradient and Hessian
- $|S_g| = \mathcal{O}(\epsilon^{-1})$, $|S_H| = \mathcal{O}(\epsilon^{-1/2})$
- Preserves convergence guarantees

*Key Challenge: Balancing second-order information accuracy with computational cost*

# Comparison with Alternative Methods

## When to Use Cubic Regularization?

| Method | Iteration | Per-step Cost | Global | Saddle |
|---|---|---|---|---|
| Gradient Descent | $\mathcal{O}(\epsilon^{-2})$ | $\mathcal{O}(d)$ | | |
| Newton's Method | Quadratic* | $\mathcal{O}(d^3)$ | | |
| L-BFGS | Superlinear* | $\mathcal{O}(d)$ | | |
| Trust-Region | $\mathcal{O}(\epsilon^{-3/2})$ | $\mathcal{O}(d^3)$ | | |
| **Cubic Reg.** | $\mathcal{O}(\epsilon^{-3/2})$ | $\mathcal{O}(d^3)$ | | |
| **Sub-sampled CR** | $\mathcal{O}(\epsilon^{-3/2})$ | $\mathcal{O}(nd)$ | | |
| **SSCN** | $\mathcal{O}(\epsilon^{-3/2})$ | $\mathcal{O}(d\tau^2)$ | | |

*Local convergence only; requires line search/trust-region for globalization

**Best Use Cases:**

- Non-convex problems with saddle points (neural networks, tensor decomposition)
- High-precision requirements (scientific computing)
- Moderate dimension ($d < 10^5$) or with subspace sampling

# Algorithmic Implementation

## ARC with Inexact Subproblem Solving

**Solving the Cubic Subproblem:**
The subproblem is equivalent to finding $s$ such that:

$$(\nabla^2 f(x_k) + \lambda I)s = -\nabla f(x_k)$$

where $\lambda = \frac{M}{2}\|s\|$.

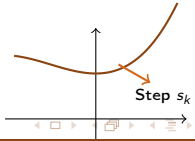
**Practical Approaches:**

1. **Lanczos method**: Reduce to tridiagonal system
2. **Krylov subspace**: Dimension-free convergence
3. **Exact solution**: For small $d$, direct eigenvalue decomposition

**Stopping Criterion:**

### Implementation Details

- **Language**: Python with NumPy/SciPy
- **Linear Algebra**: LAPACK for small $d$, iterative for large
- **Sub-sampling**: Uniform sampling without replacement
- **Parameters**: $\eta_0 = 0.1$, $\eta_1 = 0.7$, $\gamma_0 = 2$, $\gamma_1 = 0.5$



# Experimental Setup

## Datasets and Evaluation Protocol

**Datasets (LIBSVM):**

- **a9a**: 32,561 samples, 123 features
  - Census income classification
  - Binary: $> 50K$ vs $\leq 50K$
- **covtype**: 581,012 samples, 54 features
  - Forest cover type prediction
  - Binary: class 1 vs. rest

**Loss Functions:**

1. Logistic Regression + $\ell_2$ reg.
2. Non-convex: $\lambda \sum_j \frac{w_j^2}{1+w_j^2}$

**Methods Compared:**

- Gradient Descent (with line search)
- L-BFGS (memory=10)
- Newton's Method (Cholesky)
- ARC (Adaptive Cubic Regularization)
- Sub-sampled CR (SCR)

**Evaluation Metrics:**

- Gradient norm vs. iterations
- Wall-clock time
- Test accuracy
- Averaged over 5 random runs

# Preliminary Results

## Expected Behavior and Current Progress

**Expected Convergence:**

- **ARC/SCR**: Fastest convergence in iterations
- **Newton**: Similar but unstable on non-convex
- **L-BFGS**: Good balance, may stagnate at saddles
- **GD**: Slowest, especially on ill-conditioned covtype

**Computational Trade-offs:**

- ARC: Fewer iterations, higher per-step cost

**Current Status**

- ☐ ARC implementation completed
- ☐ Sub-sampled CR in progress
- ☐ Dataset preprocessing done
- ☐ Baseline methods (GD, L-BFGS) running
- ☐ Convergence plots generation

**Next Steps:**

1. Complete SCR implementation
2. Run full experimental suite (5 runs avg.)
3. Generate convergence figures

# Summary and Outlook

## Key Takeaways

**Key Messages:**

1. Cubic regularization provides **optimal second-order complexity** with **global convergence**
2. Modern variants (subspace, stochastic) enable **scalability** to ML problems
3. ARC framework eliminates manual parameter tuning
4. Recent advances (AdaCubic 2024) show promise for deep learning

**Current Challenges:**

- GPU implementation for large-scale neural networks

### Ongoing Work

- Complete experimental evaluation on a9a and covtype
- Compare with PyTorch/TensorFlow optimizers
- Investigate non-convex regularization effects
- Analyze sensitivity to hyperparameters

Theory → Algorithms → Applications

# References I


- Y. Nesterov and B.T. Polyak, "Cubic regularization of Newton method and its global performance," *Mathematical Programming*, 108(1):177–205, 2006.
- C. Cartis, N.I.M. Gould, and P.L. Toint, "Adaptive cubic regularisation methods for unconstrained optimization. Part I: motivation, convergence and numerical results," *Mathematical Programming*, 127(2):245–295, 2011.
- C. Cartis, N.I.M. Gould, and P.L. Toint, "Adaptive cubic regularisation methods for unconstrained optimization. Part II: worst-case function- and derivative-evaluation complexity," *Mathematical Programming*, 130(2):295–319, 2011.
- J.M. Kohler and A. Lucchi, "Sub-sampled cubic regularization for non-convex optimization," *ICML*, 2017.
- Y. Zhou et al., "Convergence of cubic regularization for nonconvex optimization under KŁ property," *NeurIPS*, 2018.
- J. Zhao et al., "Cubic regularized subspace Newton for non-convex optimization," *arXiv:2406.16666*, 2024.

# References II


I. Tsingalis et al., "AdaCubic: An adaptive cubic regularization optimizer for deep learning," *OpenReview*, 2024.

N. Doikov and Y. Nesterov, "First and zeroth-order implementations of the regularized Newton method with lazy approximated Hessians," *arXiv:2309.02412*, 2023.

# Thank You!

Questions?

*Contact: your.email@university.edu*
*Code: github.com/yourusername/cubic-regularization*